

Leveraging LDP for High-Trust OpenSearch UBI

Tuning search when you can't store the queries

Jeff Zemerick

OpenSearchCon NA
September 24, 2026

About me

- Independent consultant at Mountain Fog
- OpenSearch UBI and opensearch-migrations maintainer
- Apache Software Foundation member, OpenNLP PMC Chair
- Works in search, NLP, and privacy

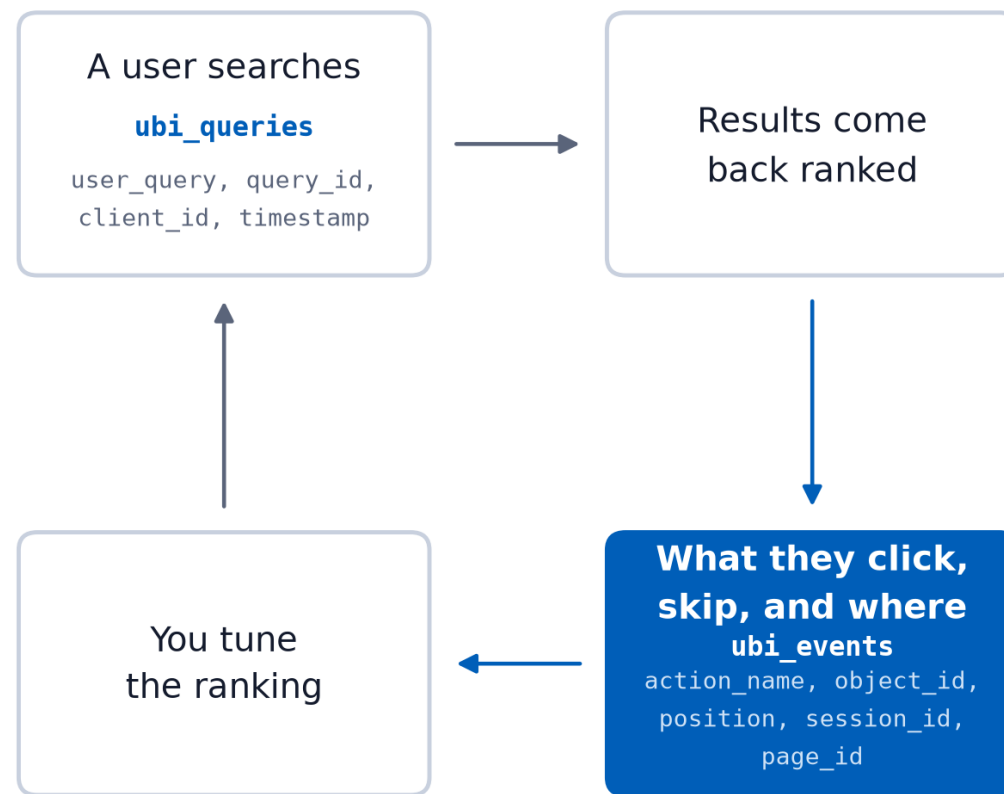
jeffzemerick.dev

PART 1

The problem

To tune search you have to see what users do

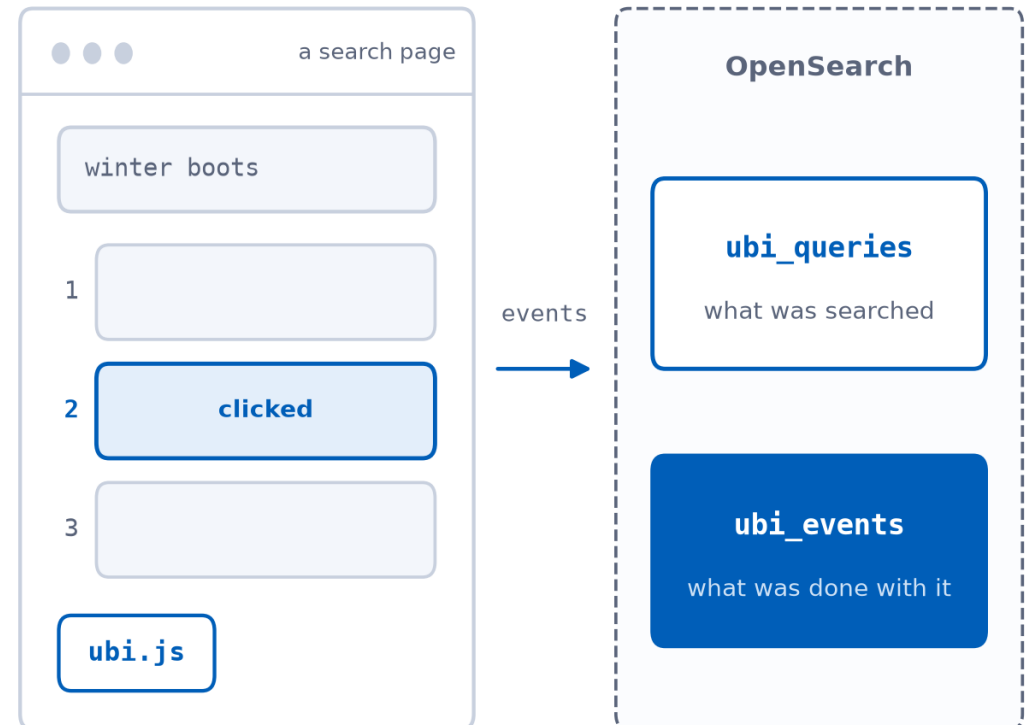
- Relevance is measured
- What queries, what's clicked, and where
- Without measurement you are guessing
- Learning-to-rank can't be trained on guesses



OpenSearch UBI exists for this

- `ubi_queries`: `user_query`, `query_id`, `client_id`, `timestamp`
- `ubi_events`: `action_name`, `object_id`, `position`, `session_id`, `page_id`
- `ubi.js` collects interactions in the browser and ships them to the cluster
- This is what we use to tune search

For examples and code, go to ubisearch.dev

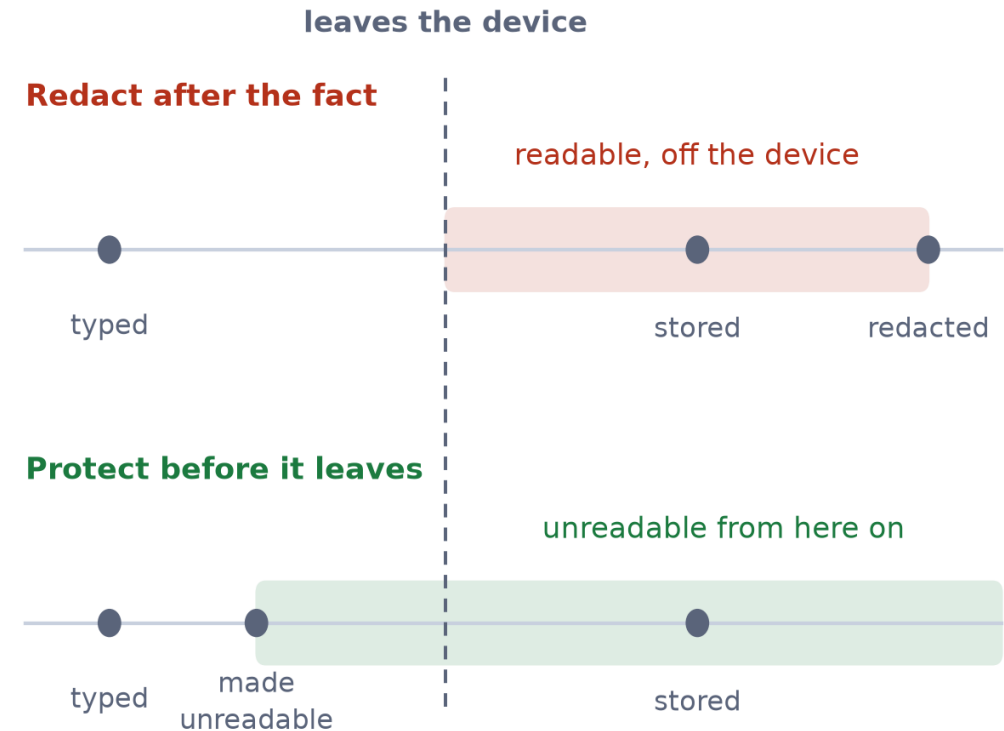


But what about when we don't know the user queries?

- user_query is free text and can contain *anything*
- Telling users not to enter PII or PHI does not stop them
- client_id and session_id make it linkable across time
- Zero-trust becomes a blocker to relevance tuning
- This applies to any behavioral data, not just search tuning

Redaction is reactive and sometimes too late

- You collect the PII first, then try to remove it
- Leaked PII is discovered later
- A free-text query has no schema to redact against
- The fix has to be that the sensitive value never leaves the device readable



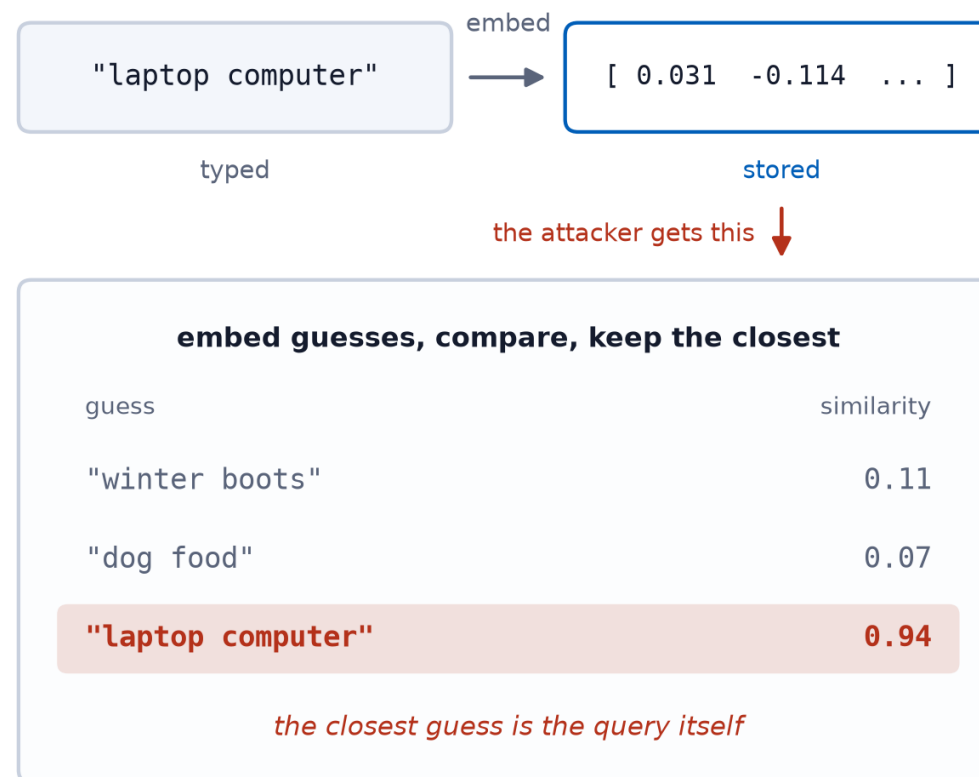
Still need to redact data at rest? philterd.ai

PART 2

Embeddings are not anonymization

The obvious first idea

- "We will not store the text. We will store the embedding."
- It's numbers, not words
- It is not private, just a lossy encoding
- Intent can be recovered



The same attack on both vectors

The attacker looks up the nearest documents to whatever vector they intercepted

Attacker intercepts	Their best guess	Distance
The raw vector	Laptop	0.2092
The noised vector	Gaming Console	4.8870

Query "laptop computer" at epsilon 1.2, from notebook section 9

The raw vector gives up the intent

- Query: "laptop computer"
- Attacker inverts the raw vector, top hit: Laptop
- Intent was leaked even though no text was stored
- **Storing embeddings is not a privacy control**

PART 3

The fix: Local Differential Privacy

Two places to add the noise

Differential privacy comes in two shapes, and the difference is who you trust

	Central DP	Local DP
Noise is added	After collection	On the device
Who sees the raw data	A trusted curator	No one
Who uses it	The 2020 US Census	Apple emoji trends

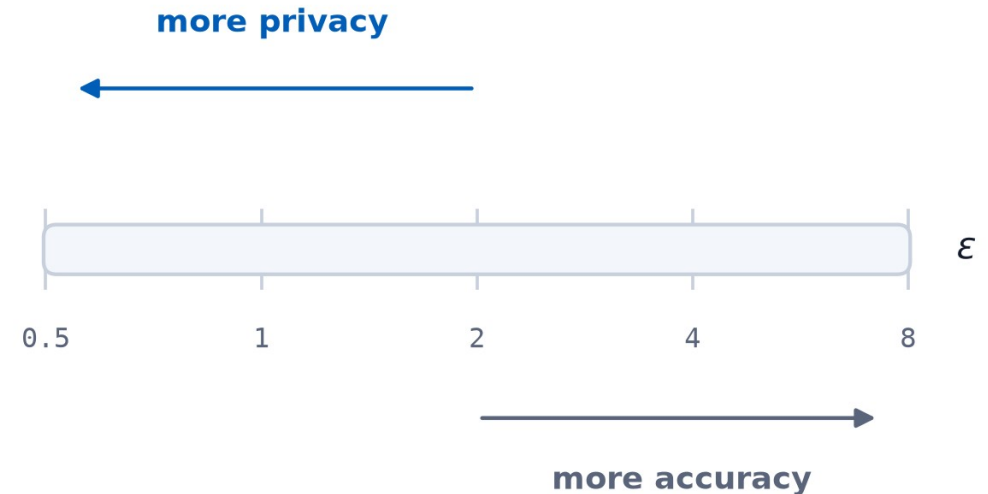
The Census Bureau sees every raw response, then noises what it publishes.

Apple never sees your emoji, only a noised count from every phone.

Apple, Learning with Privacy at Scale, 2017: machinelearning.apple.com

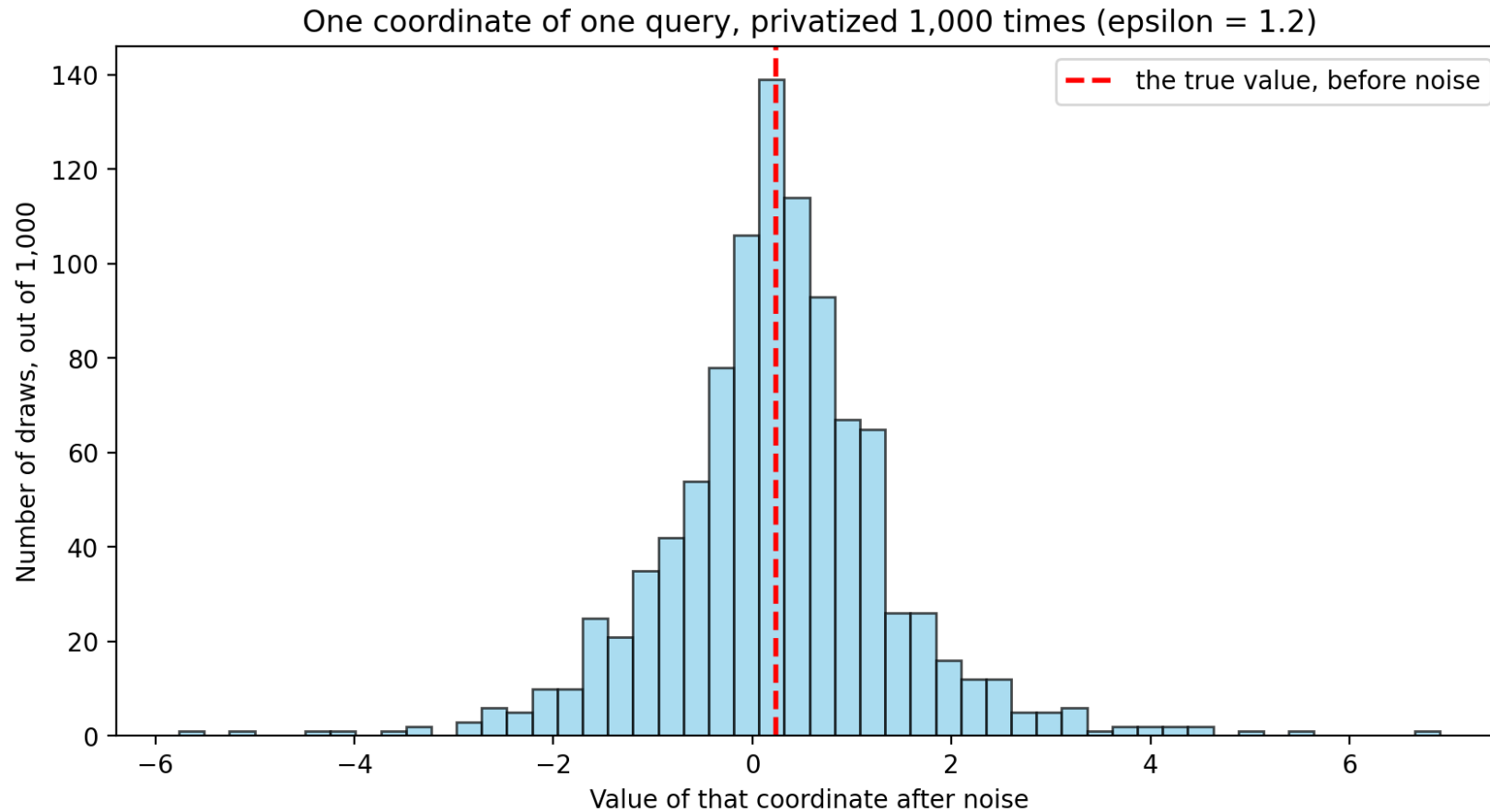
Local Differential Privacy

- The vector was never the problem, storing it readable was
- Add calibrated noise to the query vector on the device
- Epsilon is the dial, and lower epsilon means more noise and more privacy
- The user's query is not stored so there is nothing to redact



More on local differential privacy: en.wikipedia.org/wiki/Local_differential_privacy

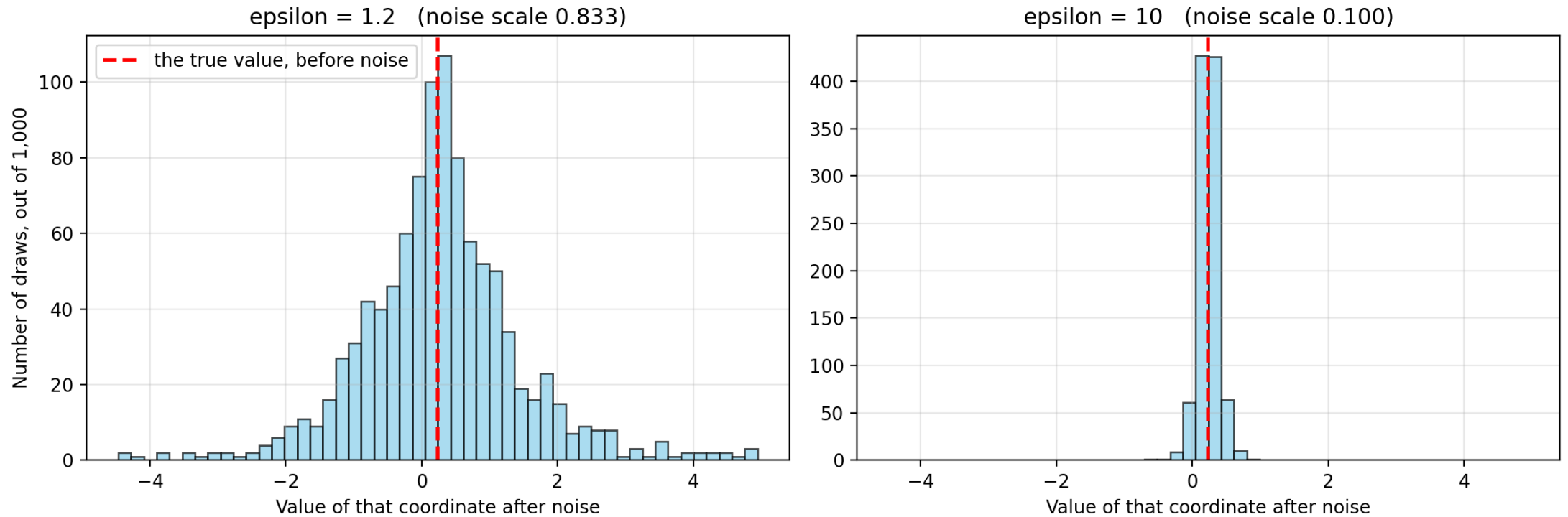
Verification 1: the noise



A biased mechanism would put the peak off the red line, and the spread is the privacy.

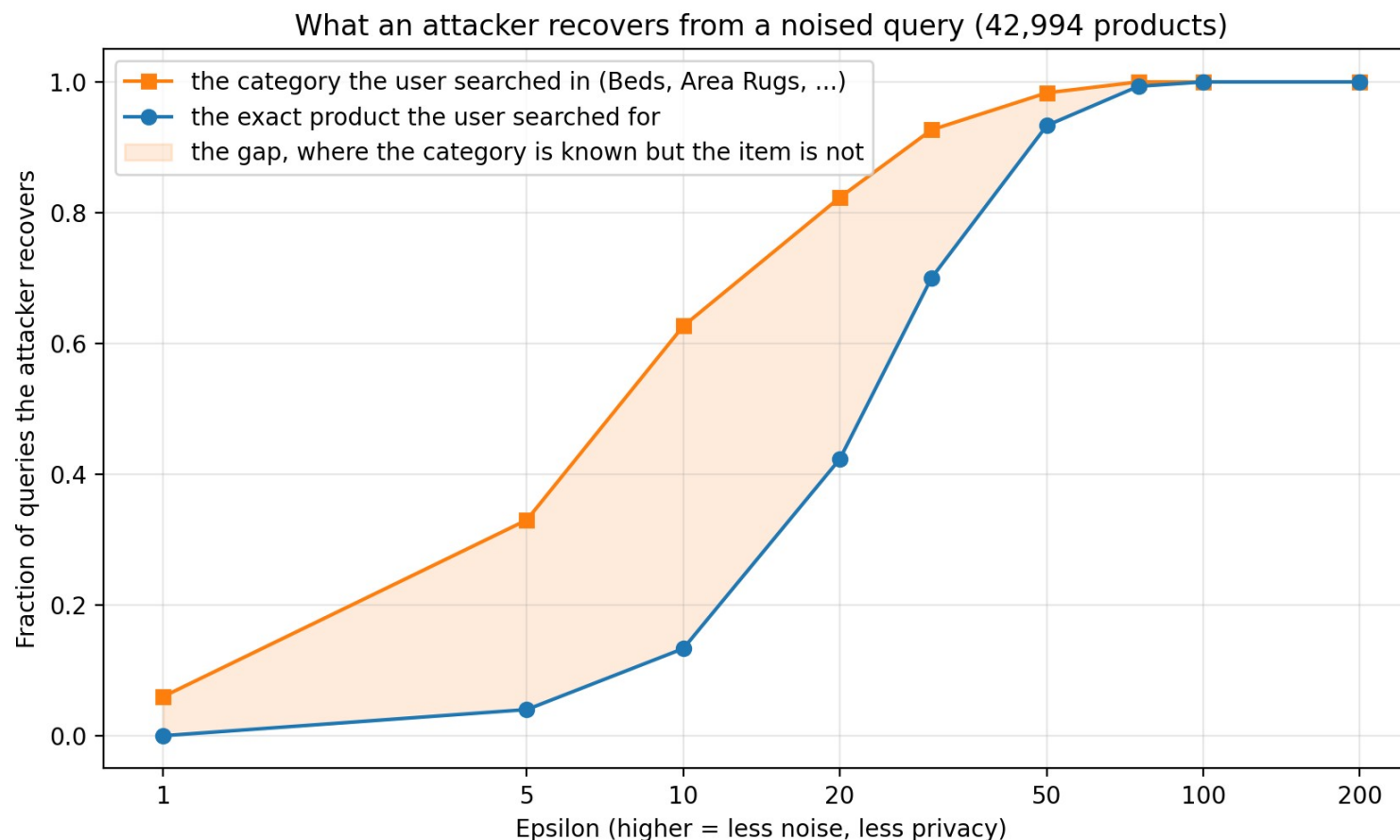
A higher epsilon is a narrower spread

Same coordinate, same 1,000 draws, two privacy settings



Both panels share one x axis. At epsilon 10 almost every draw lands on the red line.

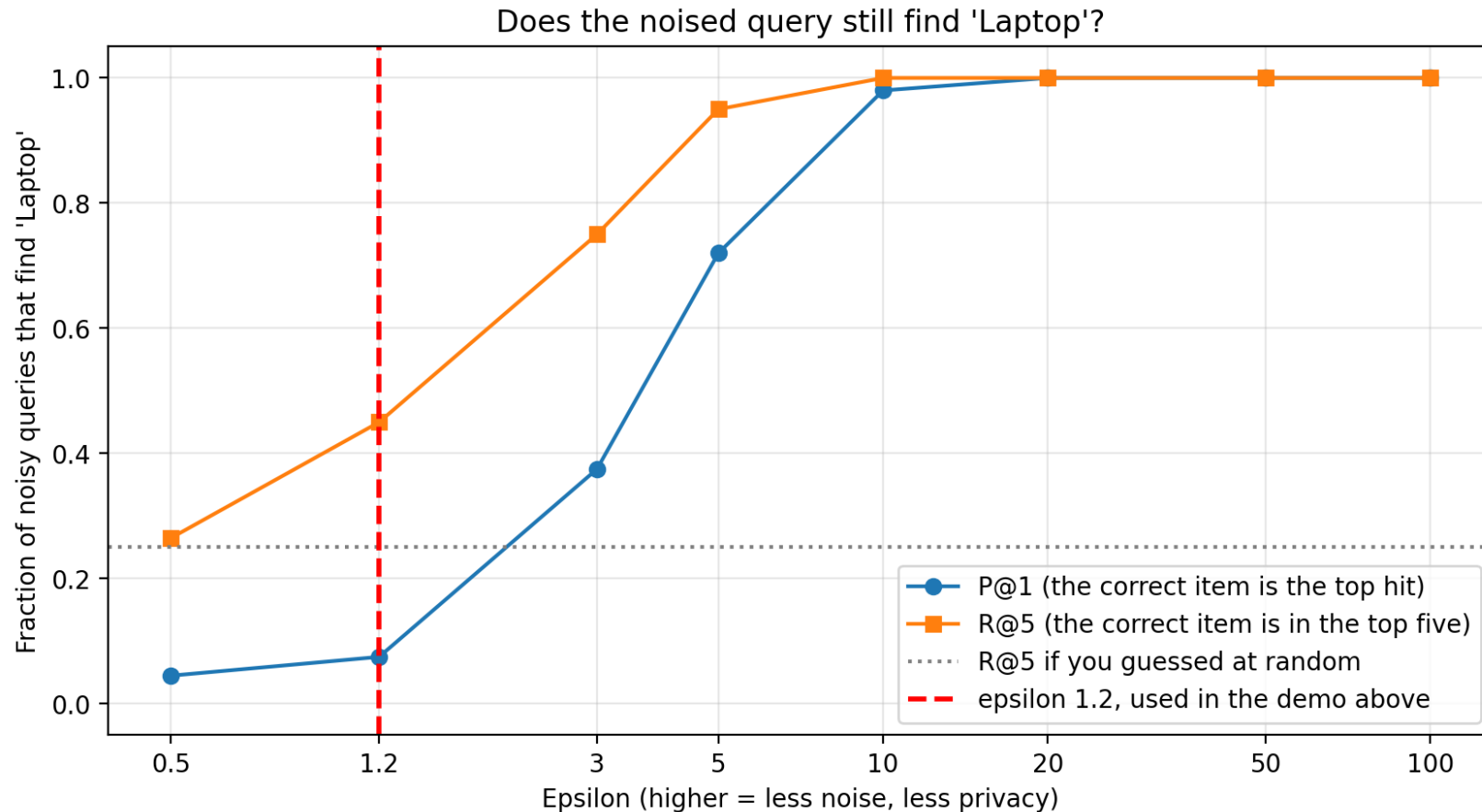
Verification 2: measure an actual attacker



The category leaks well before the item does.

Data: WANDS, Wayfair product search relevance (ECIR 2022), MIT licensed: github.com/wayfair/WANDS

Epsilon 1.2 is barely better than guessing

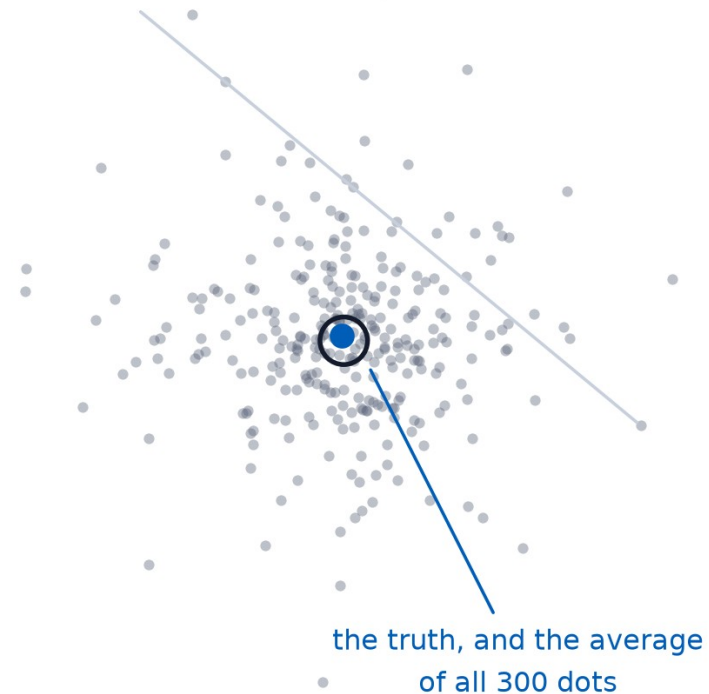


The red line is where the Part 2 demo ran. Utility only comes back where the attacker wins too.

You lose the person and keep the pattern

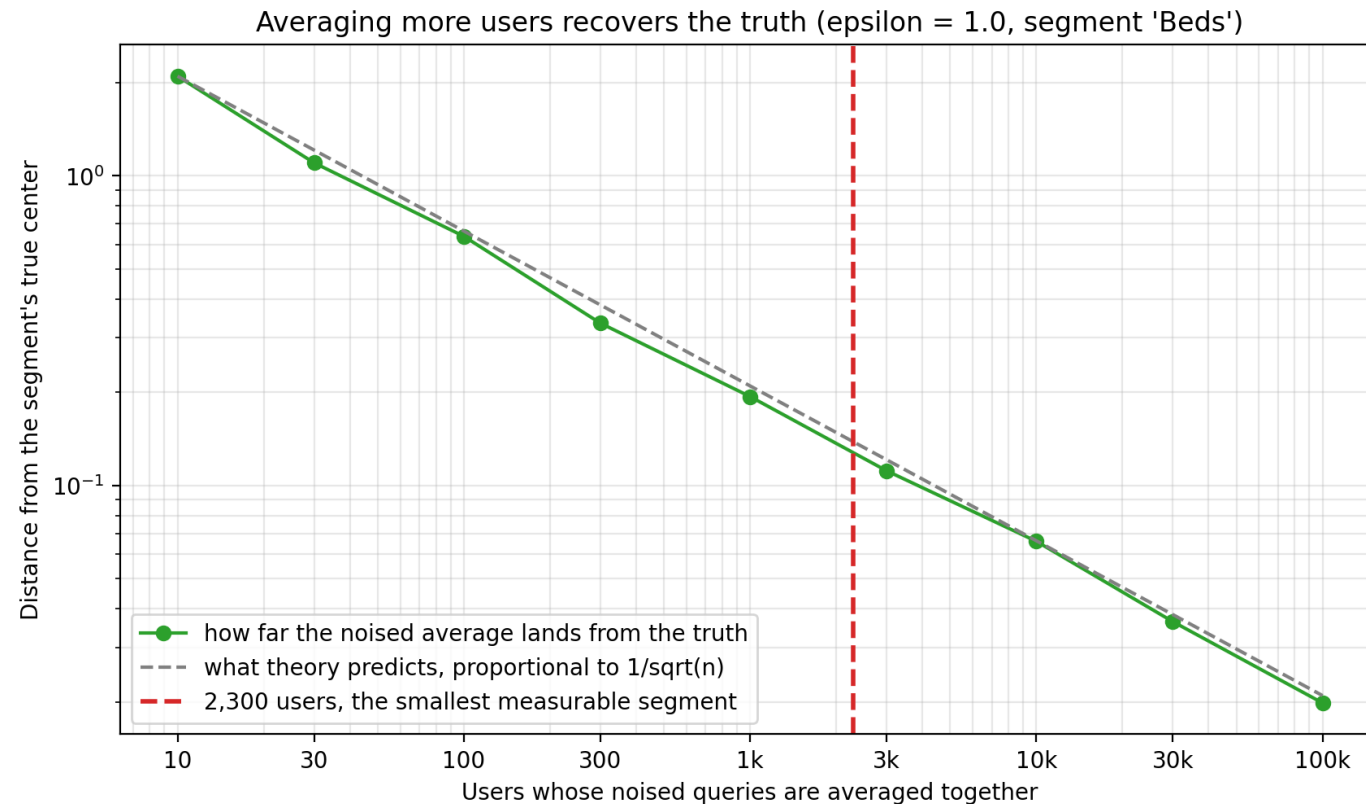
- The noise cancels when you average over many users
- One person's query is unrecoverable
- The trend across ten thousand people is not
- That trend is all UBI needs to tune relevance

one user's noised query,
useless on its own



Error falls as $1 / \sqrt{n}$

Average the noised queries of n users in one segment, then see how far that average lands from the truth.



The red line is the smallest segment you can measure.

How big a segment has to be

- Error drops below the spread of the index at roughly 2,300 users
- That is at epsilon 1, and halving epsilon needs four times the users
- Segments with thousands of users are measurable under LDP
- Segments with dozens are not measurable
- Your head terms work, but your long tail does not

So how do you pick epsilon

- Start at epsilon 1, then measure
- Pick it for the privacy you need, because utility also depends on the number of users
- Segments too small? Add users or widen them, and leave epsilon alone

LDP does not cost you your analytics.

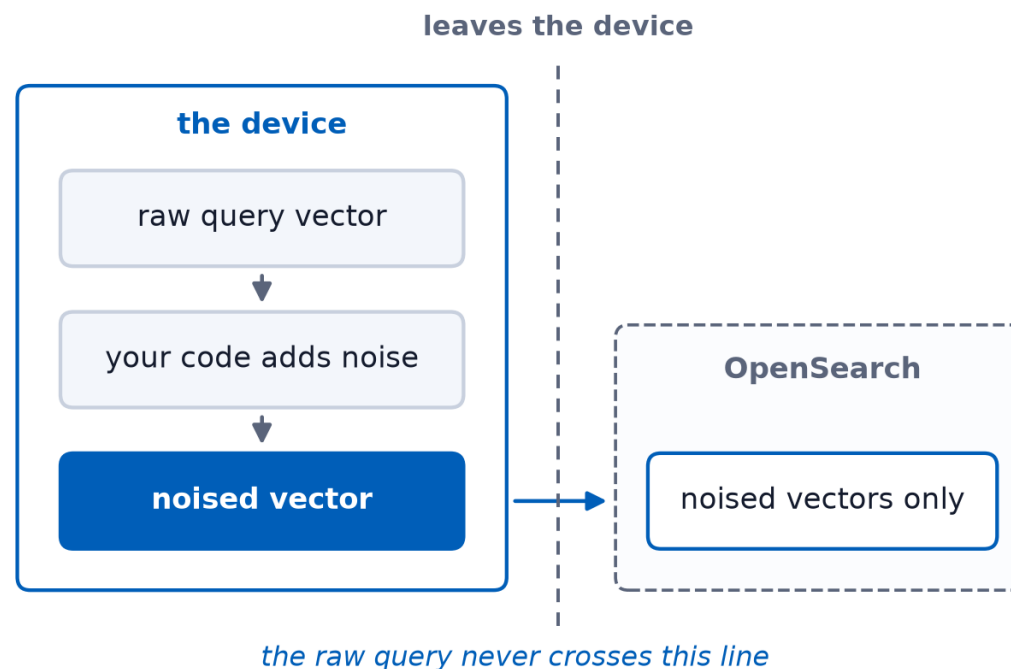
It costs you the ability to ask about one person.

PART 4

Back in OpenSearch

Where the noise gets injected

- On the device, before `ubi.js` sends the event
- Aggregation happens at query time over the noised data
- Nothing downstream needs to be trusted, because nothing downstream has the original
- You set epsilon in the client you ship



An upcoming UBI RFC will bring this capability into `ubi.js`

What changes in ubi_queries

today

```
{  
  "query_id": "q-8f31c4",  
  "client_id": "c-2f91",  
  "timestamp": "2026-09-24T10:12:03Z",  
  "user_query": "solid wood platform bed"  
}
```



with LDP

```
{  
  "query_id": "q-8f31c4",  
  "client_id": "c-2f91",  
  "timestamp": "2026-09-24T10:12:03Z",  
  "user_query": "",  
  "query_attributes": {  
    "noised_query_vector": [0.03, -1.82, ...],  
    "epsilon": 1.0  
  }  
}
```

UBI supports this today through query_attributes, and the RFC will propose making it first-class

What your dashboards can still compute

- Aggregate query intent per segment
- Demand trends and shifts over time
- Segment-level training signal for learning-to-rank
- What they cannot do is show you one user's session

Training learning-to-rank on noised queries

The labels were never protected, so what changes is what you key them on

	Classic LTR	Under LDP
Labels	Clicks	Clicks
Keyed on	The query	The segment
Query features	BM25 and term matches	The segment centroid

ubi_events still shows which results were clicked

Intent-based ranking profiles: for bed shoppers, boost these attributes

The limitations

- The category leaks and that is worse for healthcare than furniture
- Epsilon values don't transfer between indexes
- Long tail queries stay unmeasurable
- This protects `user_query` but `ubi_events` still holds clicks and ids in the clear

Takeaways

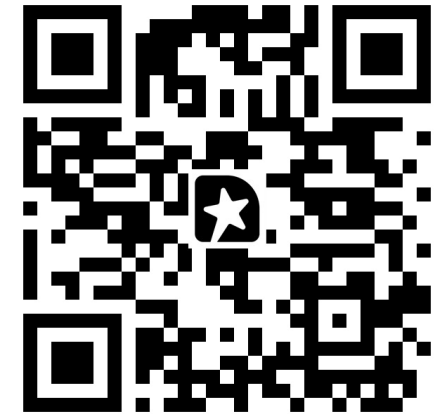
- Storing embeddings instead of text is not privacy
- Keeping sensitive data out of the index beats redacting it later
- Verify the mechanism by testing it, not by trusting it
- You trade individual resolution for population accuracy but that's what relevance tuning needs

Questions

github.com/jzonthemtn/ldp-for-search

Jeff Zemerick

OpenSearchCon NA
September 24, 2026



Leveraging LDP for High-Trust OpenSearch UBI

Session feedback